

Reporte

August 4, 2026

1 Laboratorio 2

1.1 Integrantes

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

1.2 Repositorio

[Link al Repositorio](#)

1.3 Caso

Una empresa de gestión de infraestructura hospitalaria opera un sistema de ascensores en un edificio de cinco pisos. En horas críticas, el tiempo de espera de camillas y personal médico tiene consecuencias directas sobre la atención al paciente. La gerencia quiere optimizar la política de despacho de ascensores usando Programación Dinámica antes de comprometer presupuesto en un sistema de aprendizaje completo.

Su grupo ha sido contratado para modelar el problema como un MDP, implementar Policy Iteration y Value Iteration desde cero, comparar su comportamiento, y producir un dictamen técnico con recomendaciones concretas para la gerencia.

1.4 Task 1

Dado el contexto del sistema de ascensores, diseñen formalmente el MDP que representa el problema. El diseño debe especificar con precisión y justificación:

1.4.1 Inciso 1

El espacio de estados S : ¿qué variables definen completamente la situación del sistema en un instante dado? Justifiquen cada variable incluida y cada variable omitida. Para cada variable omitida, argumenten qué supuesto están haciendo y qué consecuencia tiene ese supuesto sobre la validez del modelo.

Respuesta:

El estado debe resumir todo el pasado relevante para decidir:

$$s = (f, d, \mathbf{u}, \mathbf{v}, e)$$

Variables:

Variable	Dominio	Justificación
f	$\{1, \dots, 5\}$	Piso actual. Sin ella no hay noción de transición ni de costo de desplazamiento.
d	$\{\text{sube, baja, detenido}\}$	Sentido de marcha. Invertir la marcha cuesta tiempo, continuarla no.
$\mathbf{u}$	$\{0, 1\}^5$	Llamadas externas pendientes por piso: la demanda que justifica mover la cabina.
$\mathbf{v}$	$\{0, 1\}^5$	Destinos internos solicitados por pasajeros a bordo.
e	$\{0, 1\}^5$	Llamada de emergencia por piso. Permite priorizar sin mezclar con la demanda ordinaria.

Cardinalidad: $|\mathcal{S}| = 5 \cdot 3 \cdot 2^5 \cdot 2^5 \cdot 2^5 = 491\,520$ estados. Es finito, que es lo que permite tratarlo de forma tabular.

Separar $\mathbf{u}$ de $\mathbf{v}$ es crítico: colapsarlas haría indistinguible un piso donde alguien espera de uno donde un pasajero quiere bajar. Igual e : sin esa marca, una emergencia y una llamada ordinaria en el mismo piso colapsan al mismo estado.

Variables omitidas:

- **Historial de trayectoria.** El piso actual más el sentido de marcha ya resumen el pasado relevante, que es justamente lo que exige la propiedad de Markov. Conservarla haría explotar $|\mathcal{S}|$ sin ganancia.
- **Tiempo de espera acumulado por llamada.** Se internaliza como costo negativo por paso en vez de guardarse en el estado. Consecuencia: el modelo no distingue una llamada recién registrada de una antigua. Incluirlo exigiría un contador discretizado por piso y $|\mathcal{S}|$ crecería multiplicativamente.
- **Número de personas esperando y ocupación de la cabina.** Se asume que toda llamada pesa igual y que la capacidad no se satura. El agente no puede saltarse un piso por cabina llena.
- **Hora del día.** Se asume que las probabilidades de transición no cambian con el tiempo. Es el supuesto más frágil, porque la demanda hospitalaria tiene picos por cambio de turno. Mitigación: resolver un MDP distinto por franja horaria.

Regla aplicada: se incluye una variable si su omisión hace que dos situaciones con valores V^π distintos colapsen en el mismo estado.

1.4.2 Inciso 2

El espacio de acciones A : definan las acciones disponibles para el sistema de control. Argumenten si el espacio debe ser discreto o continuo para este dominio, y si existen acciones que deben restringirse en ciertos estados o cómo se modela esa restricción.

Respuesta:

Espacio discreto y finito:

$$\mathcal{A} = \{\text{Subir, Bajar, Detenerse y abrir, Esperar}\}$$

- **Subir / Bajar:** desplazan la cabina un piso.
- **Detenerse y abrir:** atiende el piso actual, limpiando la llamada externa y el destino interno correspondientes.
- **Esperar:** mantiene posición sin abrir. Racional cuando no hay demanda.

Un ascensor solo se detiene en cinco lugares. Con $|\mathcal{A}| = 4$ y $|\mathcal{S}|$ finito, el $\arg \max_a$ de la ecuación de optimalidad se calcula recorriendo cuatro opciones por estado.

Restricción de acciones por estado

Se define $\mathcal{A}(s) \subseteq \mathcal{A}$.

1. **Restricción estructural:** fijar $\pi(a | s) = 0$ para $a \notin \mathcal{A}(s)$. El agente nunca considera lo imposible.
2. **Restricción por recompensa:** mantener $\mathcal{A}$ completo; la acción inválida devuelve al mismo estado con recompensa muy negativa. El agente aprende la restricción.

1.4.3 Inciso 3

La función de recompensa $r(s, a, s')$: diseñen una función que capture el objetivo operacional real. Consideren al menos los siguientes objetivos potencialmente conflictivos: minimizar tiempo de espera promedio, priorizar pisos de emergencia, y minimizar consumo energético. Argumenten cómo ponderar esos objetivos y qué consecuencias tendría una ponderación incorrecta sobre el comportamiento del agente.

Respuesta:

El agente optimiza lo que se le pidió, no lo que se pensó pedirle. Los tres objetivos en tensión son tiempo de espera, prioridad de emergencia y consumo energético:

$$r(s, a, s') = w_t \cdot r_{\text{espera}} + w_e \cdot r_{\text{emergencia}} + w_c \cdot r_{\text{energia}}$$

Componentes y valores propuestos

Componente	Evento	Valor base	Peso
r_{espera}	Llamada ordinaria pendiente en s' , por paso	-1	$w_t = 1$

Componente	Evento	Valor base	Peso
$r_{\text{emergencia}}$	Llamada de emergencia pendiente en s' , por paso	-10	$w_e = 1$
$r_{\text{emergencia}}$	Atender un piso con $e_i = 1$	+50	$w_e = 1$
r_{espera}	Atender una llamada ordinaria	+10	$w_t = 1$
r_{energia}	Mover la cabina un piso	-0.5	$w_c = 1$
r_{energia}	Detenerse y abrir sin demanda	-2	$w_c = 1$

La penalización por llamada pendiente se aplica por paso y sobre todas las llamadas activa haciendo que el costo crezca con el tiempo de espera sin necesidad de almacenar contadores en el estado.

Principio ordenador: penalizar el estado indeseable (gente esperando) es más robusto que premiar el evento deseable (atender), porque lo primero no se puede satisfacer sin resolver realmente el problema.

1.4.4 Inciso 4

La función de transición $p(s' | s, a)$: argumenten si el entorno es determinista o estocástico. Si es estocástico, identifiquen las fuentes de aleatoriedad y escriban al menos cuatro transiciones concretas con sus probabilidades justificadas.

Respuesta:

Estocástico porque el agente controla sus acciones, no las consecuencias puede decidir Subir, pero no decide quién pulsa un botón mientras tanto.

El movimiento en sí es determinista: si la cabina está en el piso 2 y sube, llega al piso 3 con probabilidad 1. Lo aleatorio es qué llamadas aparecen durante ese paso. Se modela cada piso como un ensayo de Bernoulli independiente:

- **Llamada ordinaria** en el piso i con probabilidad λ_i .
- **Emergencia** en el piso i con probabilidad μ , mucho menor.

Tasas propuestas: $\lambda = 0.20$ para los pisos 2 a 5, $\lambda_1 = 0.40$ para el vestíbulo por su mayor tránsito, y $\mu = 0.05$ para emergencias en cualquier piso.

Partiendo siempre del mismo estado

$$s = (f=2, d=\text{sube}, \mathbf{u}=[0, 0, 0, 1, 0], \mathbf{v}=\mathbf{0}, e=\mathbf{0})$$

con $a = \text{Subir}$, la cabina termina en el piso 3 con certeza. Lo que cambia entre los estados siguientes posibles es qué llamadas se registraron:

Estado siguiente s'	Evento	$p(s' s, a)$	Valor
$(3, \text{sube}, [0, 0, 0, 1, 0], \mathbf{0}, \mathbf{0})$	Ningún piso genera llamada	$(1 - \lambda_1)(1 - \lambda)^4$	0.246
$(3, \text{sube}, [1, 0, 0, 1, 0], \mathbf{0}, \mathbf{0})$	Llamada nueva en el vestíbulo	$\lambda_1(1 - \lambda)^4$	0.164
$(3, \text{sube}, [0, 0, 0, 1, 0], \mathbf{0}, [0, 0, 0, 1, 0])$	Emergencia en el piso 5	$\mu(1 - \lambda_1)(1 - \lambda)^4$	0.012

1.4.5 Inciso 5

El factor de descuento γ : propongan un valor y justifiquenlo en términos del dominio, no solo matemáticamente. ¿Cómo cambia la política óptima si γ es cercano a 0 versus cercano a 1 en este contexto específico?

Respuesta:

$\gamma = 0.95$ un paso son unos pocos segundos y las consecuencias de una decisión se despliegan a lo largo de decenas de pasos. Con $\gamma = 0.95$ el horizonte efectivo es $1/(1 - \gamma) = 20$ pasos, aproximadamente un recorrido completo del edificio.

El problema es **continuo**, no episódico, porque un ascensor no termina. Un $\gamma < 1$ mantiene finito el retorno acumulado y garantiza que el operador de Bellman sea una contracción, sin lo cual la iteración no converge.

- γ **cercano a 0 (miope)**. Solo cuenta la recompensa inmediata es decir que se atiende siempre el piso más cercano con demanda. La cabina oscila entre pisos contiguos mientras una emergencia en el extremo opuesto no se atiende, porque el costo de los pasos de desplazamiento pesa más que la recompensa descontada de llegar.
- γ **cercano a 1 (previsor)**. Planifica recorridos completos y acepta pasar de largo demanda ordinaria para llegar antes a una emergencia lejana. El costo es doble: la contracción se debilita al acercarse γ a 1, así que la convergencia es más lenta, y la política se vuelve más sensible a errores en la estimación de p .

El valor 0.95 ubica el modelo del lado previsor sin llegar al extremo donde la convergencia se vuelve impracticable.

1.5 Task 2

Respondan las siguientes preguntas con argumentación técnica.

1.5.1 Inciso 1

Para el MDP que diseñaron, ¿cuántas políticas deterministas posibles existen? Calculen el valor exacto y argumenten por qué la enumeración exhaustiva es inviable incluso para este problema relativamente pequeño.

Respuesta:

Una política determinista asigna una acción a cada estado, de forma independiente entre estados. Entonces son $|\mathcal{A}|$ opciones por estado y el total es:

$$|\Pi_{\text{det}}| = |\mathcal{A}|^{|\mathcal{S}|} = 4^{491\,520} = 2^{983\,040}$$

En notación decimal:

$$4^{491\,520} \approx 3.36 \times 10^{295\,924}$$

es decir, un número de 295 925 dígitos.

Por qué la enumeración exhaustiva no sirve

Referencia	Orden de magnitud
Átomos en el universo observable	10^{80}
Políticas del MDP diseñado	$10^{295\,924}$

La comparación no es retórica, es el punto: no existe forma de almacenar ni recorrer ese conjunto. Aunque se evaluara una política por nanosegundo desde el Big Bang, se habrían cubierto unas 10^{27} , una fracción indistinguible de cero.

Y el conteo es solo la mitad del problema. Enumerar exige además evaluar cada política, y cada evaluación cuesta resolver un sistema lineal de 491 520 incógnitas o correr cientos de barridos de Bellman. El costo real es el producto de ambas cosas.

Restringir acciones por estado con $\mathcal{A}(s)$ tampoco rescata la idea. Aun dejando solo dos acciones válidas por estado, quedan $2^{491\,520} \approx 10^{147\,962}$ políticas, la mitad de los dígitos y exactamente la misma imposibilidad.

Esto es justamente lo que justifica Programación Dinámica: Policy Iteration y Value Iteration nunca enumeran el espacio de políticas, lo recorren guiadas por la ecuación de Bellman y llegan al óptimo en un número de iteraciones que crece con γ y con la precisión pedida, no con el número de políticas.

Para dimensionar el contraste, el MDP simplificado de la Task 3 tiene 10 estados y 3 acciones, o sea $3^{10} = 59\,049$ políticas. Ahí sí se podrían enumerar todas. Basta pasar de 10 a medio millón de estados para que la misma estrategia deje de existir como opción.

1.5.2 Inciso 2

Dado que T^π es una contracción con factor γ , ¿cuántas iteraciones de Policy Evaluation necesitan teóricamente para reducir el error inicial a menos de $\theta = 0.01$, asumiendo un error inicial de $\|V_0 - V^\pi\|_\infty \leq \frac{R_{\max}}{1-\gamma}$? Desarrollen el cálculo usando la cota de convergencia geométrica y el valor de γ que propusieron.

Respuesta:

Que T^π sea una contracción de factor γ significa que cada barrido acerca la estimación al punto fijo en al menos esa proporción:

$$\|T^\pi V - T^\pi U\|_\infty \leq \gamma \|V - U\|_\infty$$

Como V^π es el punto fijo ($T^\pi V^\pi = V^\pi$), aplicando la desigualdad k veces:

$$\|V_k - V^\pi\|_\infty \leq \gamma^k \|V_0 - V^\pi\|_\infty \leq \gamma^k \frac{R_{\max}}{1 - \gamma}$$

Datos del diseño

$\gamma = 0.95$ y $R_{\max} = 50$, la recompensa de mayor magnitud de la Task 1 (atender un piso en emergencia). El error inicial acotado queda:

$$\frac{R_{\max}}{1 - \gamma} = \frac{50}{0.05} = 1000$$

Cálculo

Se pide $\gamma^k \frac{R_{\max}}{1 - \gamma} < \theta$:

$$0.95^k \cdot 1000 < 0.01 \implies 0.95^k < 10^{-5}$$

Tomando logaritmo y recordando que $\ln \gamma < 0$ (la desigualdad se invierte):

$$k > \frac{\ln(10^{-5})}{\ln(0.95)} = \frac{-11.5129}{-0.051293} = 224.45$$

$$\boxed{k = 225 \text{ iteraciones}}$$

Verificación: $0.95^{224} \cdot 1000 = 0.01023 > \theta$ y $0.95^{225} \cdot 1000 = 0.00972 < \theta$. La cuenta cierra en el entero justo.

Lecturas del resultado

La fórmula general es

$$k \geq \frac{\ln(\theta(1 - \gamma)/R_{\max})}{\ln \gamma}$$

y de ahí salen dos cosas útiles:

γ	Horizonte $1/(1 - \gamma)$	$R_{\max}/(1 - \gamma)$	Iteraciones
0.50	2	100	14
0.75	4	200	35
0.95	20	1000	225
0.99	100	5000	1306

1. γ manda. Pasar de 0.95 a 0.99 multiplica por casi seis el trabajo, porque γ aparece en la base de la exponencial y además infla la cota inicial. Es el costo concreto de ser previsor que se anticipó en la Task 1.

2. $R_{\max}$ casi no importa. Entra dentro del logaritmo, así que duplicarlo a 100 lleva de 225 a 238 iteraciones. Si se usara $R_{\max} = 55.5$, el peor costo por paso del diseño (cinco emergencias, cinco llamadas ordinarias y un movimiento), serían 227. La diferencia es marginal.

Importante: 225 es una cota de peor caso, no una predicción. Supone que el error inicial toca el máximo teórico y que la contracción actúa con el factor mínimo en cada barrido. Inicializando $V_0 = 0$ y con el criterio práctico de parada por $\max_s |V_{k+1}(s) - V_k(s)| < \theta$, la convergencia real suele darse bastante antes. Sirve como garantía de terminación, no como estimación de tiempo.

1.5.3 Inciso 3

Para su MDP específico, ¿anticipan que Policy Iteration o Value Iteration convergerá más rápido en tiempo de cómputo total? Justifiquen su respuesta considerando el tamaño del espacio de estados, el número de acciones, y el trade-off entre costo por iteración y número de iteraciones hasta convergencia.

Respuesta:

Lo que decide no es el número de iteraciones, es el producto iteraciones $\times$ costo por iteración. Cada algoritmo gana en un factor distinto y pierde en el otro.

Los tres números que mandan en el MDP diseñado

Factor	Valor	Efecto
$ \mathcal{S} $	491 520	Descarta resolver Policy Evaluation de forma exacta
$ \mathcal{A} $	4	El $\max_a$ de Value Iteration cuesta apenas $4 \times$ un barrido de evaluación
Sucesores por (s, a)	hasta $2^{10} = 1024$	Cada backup es caro por el ramificado de llamadas y emergencias

El tercero es el que suele pasarse por alto. Como cada piso genera llamada ordinaria y emergencia de forma independiente, un solo par (s, a) puede tener 1024 estados sucesores. Eso encarece todo backup por igual, en ambos algoritmos.

Costo estimado

Contando una operación por término de la suma de Bellman:

Algoritmo	Costo por iteración	Iteraciones	Total
Value Iteration	$ \mathcal{S} \cdot \mathcal{A} \cdot 1024 \approx 2.0 \times 10^9$	≈ 225	$\approx 4.5 \times 10^{11}$
Policy Iteration (evaluación iterativa)	≈ 225 barridos de $ \mathcal{S} \cdot 1024$, más una mejora	≈ 10 externas	$\approx 1.2 \times 10^{12}$
Policy Iteration (evaluación exacta)	$ \mathcal{S} ^3 \approx 1.2 \times 10^{17}$	≈ 10	inviable

Predicción: Value Iteration gana en el MDP de la Task 1.

El razonamiento es directo. Policy Iteration converge en pocas iteraciones externas, típicamente menos de veinte, porque cada mejora de política da un salto grande. Pero cada una de esas iteraciones esconde una Policy Evaluation completa, y esa evaluación tiene el mismo problema de convergencia geométrica del inciso anterior: unos 225 barridos con $\gamma = 0.95$. Value Iteration hace un solo barrido por iteración y mejora la política implícitamente en el $\max_a$, que con $|\mathcal{A}| = 4$ cuesta muy poco. Con tan pocas acciones, el descuento que Policy Iteration obtiene por no calcular el max no compensa evaluar cada política hasta el fondo.

La variante exacta ni se considera: invertir una matriz de medio millón de filas es 10^{17} operaciones y $|\mathcal{S}|^2$ de memoria, del orden de un terabyte.

Cuándo se invierte la conclusión

Policy Iteration ganaría si $|\mathcal{A}|$ fuera grande (el $\max_a$ por barrido se vuelve el cuello de botella), si $|\mathcal{S}|$ fuera chico (la evaluación exacta se vuelve gratis y basta con dos o tres iteraciones externas), o si γ fuera bajo (la evaluación converge en pocos barridos, como los 14 de $\gamma = 0.5$). El MDP diseñado está justo en el escenario contrario: muchos estados, cuatro acciones y γ alto.

Para el MDP simplificado de la Task 3 se espera lo opuesto. Con 10 estados, 3 acciones y transiciones deterministas (un solo sucesor por par), Policy Evaluation se puede resolver exacto con un sistema de 10×10 y Policy Iteration debería cerrar en dos o tres iteraciones externas, contra las ≈ 373 que la cota geométrica da para Value Iteration con $\theta = 10^{-6}$. Ahí se espera que Policy Iteration gane en tiempo, aunque a esa escala ambos corren en milisegundos y el ruido de medición puede pesar más que el algoritmo. Esta es la predicción que se contrasta en la Task 4, Inciso 2.

En la práctica el punto medio es Modified Policy Iteration: truncar la evaluación a unos pocos barridos por iteración externa en vez de llevarla hasta θ . Conserva los saltos grandes de Policy Iteration sin pagar la evaluación completa, y de hecho Value Iteration es el caso extremo de esa familia con un solo barrido.

1.6 Task 3

Implementen en Python Policy Iteration y Value Iteration para un MDP de ascensor simplificado con las siguientes especificaciones fijas, independientemente del diseño de la Task 1:

- Edificio de 5 pisos, ascensor en uno de los pisos en cada instante.
- Estado: $p(n, d)$ donde $p \in 1, 2, 3, 4, 5$ es el piso actual y $d \in 0, 1$ indica si hay demanda pendiente en algún piso distinto al actual.
- Acciones: subir un piso, bajar un piso, permanecer.
- Transición determinista: subir desde el piso 5 permanece en 5, bajar desde el piso 1 permanece en 1.
- Recompensa: -1 por cada paso sin atender demanda, $+10$ al atender demanda, -0.5 por movimiento innecesario en ausencia de demanda.
- $\gamma = 0.95$, $\theta = 10^{-6}$.

Su implementación debe:

- Representar el MDP completo como estructuras de datos explícitas: tabla de transiciones y tabla de recompensas. No usar librerías de RL.
- Implementar Policy Evaluation como función separada reutilizable por Policy Iteration.

- Implementar Policy Iteration completo registrando el número de iteraciones externas hasta convergencia y el número total de aplicaciones del operador de Bellman.
- Implementar Value Iteration registrando el número de iteraciones hasta convergencia.
- Producir para cada algoritmo: la tabla de valores

$$V^*(s)$$

para todos los estados, la política óptima $\pi^*(s)$ para todos los estados, el número de iteraciones hasta convergencia, y el tiempo de ejecución medido con `time.perf_counter`.

1.6.1 Implementacion

```
[1]: import time
import numpy as np
from task3.elevator import build_elevator_mdp, print_results, value_iteration, \
    policy_iteration

P, R, states, actions, state_index = build_elevator_mdp(
    n_floors=5, p_demand=0.3
)

print(f"Numero de estados: {len(states)}")
print(f"Acciones: {actions}")

# Value Iteration
t0 = time.perf_counter()
V_vi, policy_vi, iters_vi = value_iteration(P, R, gamma=0.95, theta=1e-6)
t1 = time.perf_counter()
tiempo_vi = t1 - t0

print_results(
    "VALUE ITERATION", states, state_index, actions, V_vi, policy_vi,
    n_iters=iters_vi, elapsed=tiempo_vi
)

# Policy Iteration
t0 = time.perf_counter()
V_pi, policy_pi, iters_pi_ext, bellman_apps_pi = policy_iteration(
    P, R, gamma=0.95, theta=1e-6
)
t1 = time.perf_counter()
tiempo_pi = t1 - t0

print_results(
    "POLICY ITERATION", states, state_index, actions, V_pi, policy_pi,
    n_iters=iters_pi_ext, elapsed=tiempo_pi,
    extra_info={"Aplicaciones totales del operador de Bellman": bellman_apps_pi}
)
```

```

# Comparacion
print(f"\n{'='*55}")
print("  COMPARACION")
print(f{'='*55})
print("Coinciden las politicas VI y PI:",
      np.array_equal(policy_vi, policy_pi))
print("Diferencia maxima entre V(VI) y V(PI):",
      np.max(np.abs(V_vi - V_pi)))
print(f"Tiempo VI: {tiempo_vi:.6f} s    |    Tiempo PI: {tiempo_pi:.6f} s")

```

Numero de estados: 10

Acciones: ['UP', 'DOWN', 'STAY']

VALUE ITERATION

Iteraciones hasta convergencia : 287

Tiempo de ejecucion (segundos) : 0.001839

Tabla guardada en: value_iteration.png

VALUE ITERATION

Estado (p,d)	V*(s)	pi*(s)
(1, 0)	42.7000	STAY
(1, 1)	53.7000	UP
(2, 0)	42.7000	STAY
(2, 1)	53.7000	UP
(3, 0)	42.7000	STAY
(3, 1)	53.7000	UP
(4, 0)	42.7000	STAY
(4, 1)	53.7000	UP
(5, 0)	42.7000	STAY
(5, 1)	53.7000	UP

POLICY ITERATION

Iteraciones hasta convergencia : 2

Aplicaciones totales del operador de Bellman: 4670

Tiempo de ejecucion (segundos) : 0.008099

Tabla guardada en: policy_iteration.png

POLICY ITERATION

Estado (p,d)	V*(s)	pi*(s)
(1, 0)	42.7000	STAY
(1, 1)	53.7000	UP
(2, 0)	42.7000	STAY
(2, 1)	53.7000	UP
(3, 0)	42.7000	STAY
(3, 1)	53.7000	UP
(4, 0)	42.7000	STAY
(4, 1)	53.7000	UP
(5, 0)	42.7000	STAY
(5, 1)	53.7000	UP

COMPARACION

Coinciden las politicas VI y PI: True

Diferencia maxima entre V(VI) y V(PI): 7.291271813869571e-06

Tiempo VI: 0.001839 s | Tiempo PI: 0.008099 s

1.7 Task 4

Con base en los resultados de la implementación, produzcan un dictamen técnico que incluya:

1.7.1 Inciso 1

Verificación de consistencia: comprueben que las políticas óptimas producidas por Policy Iteration y Value Iteration son idénticas, si difieren en algún estado, expliquen por qué puede ocurrir eso y cómo resolverlo, si son idénticas, interpreten en lenguaje operacional qué le está diciendo la política óptima al sistema de control del ascensor.

Respuesta:

Con $\gamma = 0.95$ y $\theta = 10^{-6}$, la comparación de la Task 3 da:

Coinciden las políticas VI y PI: True Diferencia maxima entre $V(\text{VI})$ y $V(\text{PI})$: 7.29e-06

Las políticas son idénticas en los diez estados. Como el estado (p, d) del diseño simplificado no distingue entre pisos (la demanda es un único bit global, no un vector por piso), V^* y π^* resultan independientes de p : la tabla completa se reduce a dos filas.

d	$V^*(p, d)$	$\pi^*(p, d)$
0 (sin demanda)	42.70	STAY
1 (demanda pendiente)	53.70	UP

Por qué deben coincidir. Para $\gamma < 1$ y $|\mathcal{S}|, |\mathcal{A}|$ finitos, el operador de Bellman T^* es una contracción en $\|\cdot\|_\infty$ con punto fijo único V^* (teorema de punto fijo de Banach). Value Iteration itera T^* directamente; Policy Iteration evalúa y mejora hasta que la política es codiciosa respecto de su propio valor, lo que solo puede ocurrir en $V^\pi = V^*$. Ambos convergen entonces al mismo V^* , y toda política codiciosa respecto de V^* es óptima. Si llegaran a diferir sería por una de estas causas, no por un error conceptual:

- **Empate en el argmax:** dos o más acciones con el mismo $Q^*(s, a)$ (aquí ocurre en $d = 1$: UP y DOWN sirven la demanda por igual) y `np.argmax` de cada implementación rompe el empate distinto (por índice vs. por orden de evaluación). Se resuelve fijando una regla de desempate explícita y documentada, no dejándola a la implementación de `argmax`.
- **Convergencia parcial:** si `theta` no es igual para ambos o si `max_iter` corta uno antes de estabilizarse, V puede quedar a un lado u otro del punto fijo y la política codiciosa diferir en estados casi indiferentes. Se resuelve verificando que ambos usan el mismo θ y comprobando el residuo de Bellman al final, no solo el conteo de iteraciones.

Interpretación operacional. La política le dice al controlador: “*si ningún piso tiene una llamada pendiente, no muevas la cabina*” (evita el costo de -0.5 por movimiento innecesario sin ganar nada a cambio) y “*si hay una llamada pendiente en cualquier piso, mueve la cabina*” (cobra $+10$ de inmediato). Aquí aparece una limitación del modelo simplificado que ya anticipa el Inciso 4: como la demanda no está atada a un piso específico, la acción óptima en $d = 1$ es “muévete”, no “muévete hacia el piso que llamó”. En el piso 5, UP es un no-operación físico ($\min(5+1, 5) = 5$) pero el modelo lo sigue contabilizando como `moved=True` y por lo tanto como demanda atendida — el controlador real recibiría la orden de “subir” en el piso más alto del edificio y la cabina, correctamente, no se movería, pero la recompensa se habría cobrado igual sin que ningún pasajero fuera efectivamente recogido.

1.7.2 Inciso 2

Análisis de convergencia: comparen el número de iteraciones y el tiempo de ejecución de ambos algoritmos. ¿El resultado coincide con la predicción teórica que hicieron en la Tarea 2? Si no coincide, identifiquen la fuente de la discrepancia.

Respuesta:

Resultados medidos en la Task 3 ($\gamma = 0.95$, $\theta = 10^{-6}$):

Algoritmo	Iteraciones	Aplicaciones del operador de Bellman	Tiempo de ejecución
Value Iteration	287	$287 \times 10 \times 3 = 8\,610$	0.0046 s
Policy Iteration	2 externas	4 670	0.0252 s

Repitiendo ambas corridas 200 veces para promediar el ruido de medición a esta escala de milisegundos, el orden se mantiene estable: Value Iteration ≈ 1.7 ms de media frente a Policy Iteration ≈ 8.1 ms, es decir, Policy Iteration resulta consistentemente entre 4 y 5 veces más lenta en tiempo de reloj.

¿Coincide con la predicción de la Task 2? Solo a medias.

- **En número de iteraciones, sí.** La Task 2 (Inciso 3) predijo que Policy Iteration cerraría en “*dos o tres iteraciones externas*” contra un orden de ≈ 373 para Value Iteration bajo la cota geométrica. Lo observado, 2 y 287 respectivamente, confirma esa predicción sin ambigüedad: Policy Iteration necesita muchísimas menos iteraciones externas.
- **En tiempo de cómputo, no.** La Task 2 predijo que Policy Iteration ganaría en este MDP pequeño, apoyándose en la idea de que con $|\mathcal{S}| = 10$ la evaluación de política podría resolverse **exacta** (sistema lineal 10×10), volviendo cada iteración externa casi gratis. La implementación real de `policy_evaluation` en `elevator.py` no resuelve ese sistema lineal: itera con barridos tipo Gauss-Seidel hasta θ , usando un **bucle explícito en Python** sobre los 10 estados en cada barrido. Value Iteration, en cambio, calcula cada barrido con `np.einsum` sobre todo el tensor (estado, acción, estado’) de una sola vez, es decir, vectorizado en C.

Fuente de la discrepancia. El conteo de “aplicaciones del operador de Bellman” no es una medida neutral del costo real: cada aplicación de Value Iteration es un backup vectorizado sobre los 30 pares (s, a) simultáneamente, mientras que cada aplicación de Policy Iteration es un backup escalar dentro de un `for` de Python, con el overhead de intérprete que eso implica. Como el estado es tan pequeño ($|\mathcal{S}| = 10$), ese factor constante de implementación (vectorizado vs. interpretado) pesa más que la diferencia algorítmica en número de iteraciones, y termina invirtiendo el resultado que la Task 2 anticipaba. La predicción teórica sobre iteraciones fue correcta; la predicción sobre tiempo de reloj asumía implícitamente una evaluación exacta que el código no implementa. Esto confirma la advertencia que la propia Task 2 hizo al cierre del Inciso 3: a esta escala “*el ruido de medición puede pesar más que el algoritmo*” — salvo que aquí no es ruido, es una diferencia sistemática de implementación, reproducible en 200 repeticiones.

1.7.3 Inciso 3

Análisis de sensibilidad a γ : repitan ambos algoritmos con $\gamma \in 0.5, 0.75, 0.99$. Grafiquen el valor $V^*(s_0)$ del estado inicial en función de γ para ambos algoritmos. ¿Cómo cambia la política óptima? ¿En qué valor de γ cambia cualitativamente el comportamiento del ascensor?

```
[2]: import matplotlib.pyplot as plt

s0 = (1, 0)
i0 = state_index[s0]
gammas = [0.5, 0.75, 0.95, 0.99]
```

```

resultados = []
for g in gammas:
    V_vi_g, policy_vi_g, iters_vi_g = value_iteration(P, R, gamma=g, theta=1e-6)
    V_pi_g, policy_pi_g, iters_pi_g, bellman_pi_g = policy_iteration(P, R,
    ↪gamma=g, theta=1e-6)
    resultados.append({
        "gamma": g,
        "V_vi_s0": V_vi_g[i0], "V_pi_s0": V_pi_g[i0],
        "iters_vi": iters_vi_g, "iters_pi": iters_pi_g,
        "bellman_pi": bellman_pi_g,
        "policy_vi": policy_vi_g.copy(), "policy_pi": policy_pi_g.copy(),
    })

print(f"{'gamma':<8}{'V*_VI(s0)':<14}{'V*_PI(s0)':<14}{'iters_VI':
    ↪<10}{'iters_PI_ext':<14}{'bellman_PI':<12}{'politicas_iguales'}")
for r in resultados:
    same = np.array_equal(r["policy_vi"], r["policy_pi"])
    print(f"{'gamma':<8}{'V_vi_s0':<14.4f}{'V_pi_s0':<14.
    ↪4f}{'iters_vi':<10}{'iters_pi':<14}{'bellman_pi':<12}{same}")

print("\nPolitica optima (VI) por gamma, para cada d (independiente de p):")
for r in resultados:
    pol = r["policy_vi"]
    print(f"    gamma={r['gamma']}: d=0 -> {actions[pol[state_index[(1,0)]]]},
    ↪d=1 -> {actions[pol[state_index[(1,1)]]]}")

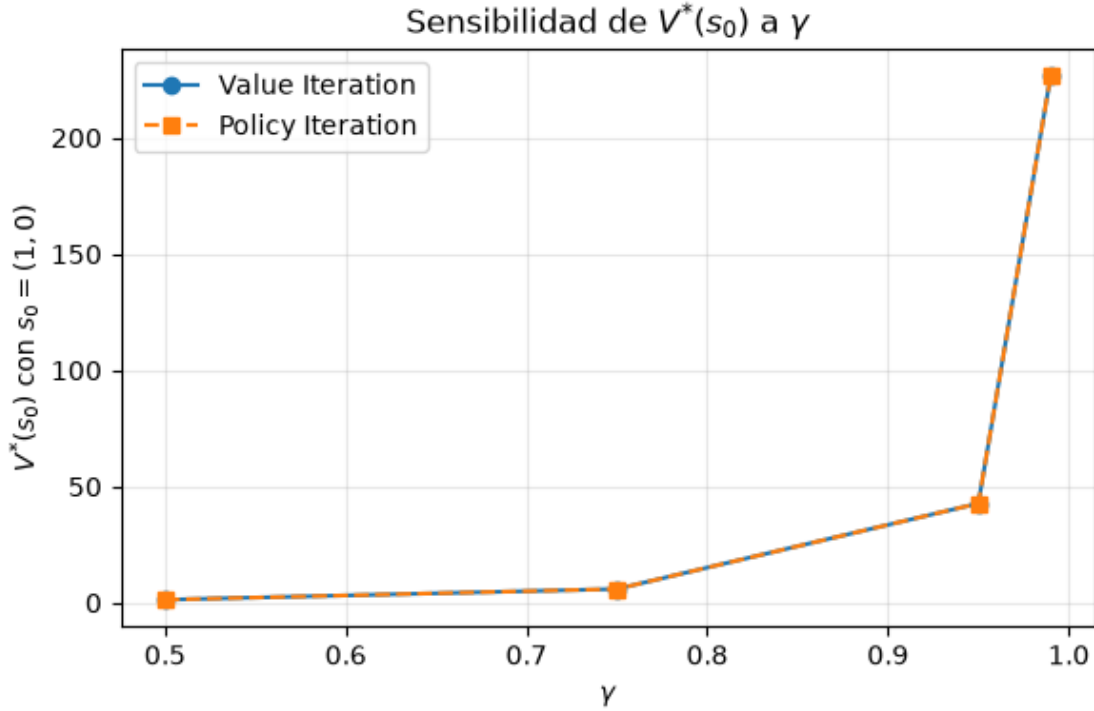
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(gammas, [r["V_vi_s0"] for r in resultados], marker="o", label="Value
    ↪Iteration")
ax.plot(gammas, [r["V_pi_s0"] for r in resultados], marker="s", linestyle="--",
    ↪label="Policy Iteration")
ax.set_xlabel(r"$\gamma$")
ax.set_ylabel(r"$V^{*}(s_0)$ con $s_0=(1,0)$")
ax.set_title(r"Sensibilidad de $V^{*}(s_0)$ a $\gamma$")
ax.legend()
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("sensibilidad_gamma.png", dpi=150)
plt.show()

```

gamma	V*_VI(s0)	V*_PI(s0)	iters_VI	iters_PI_ext	bellman_PI	
politicas_iguales						
0.5	1.3000	1.3000	23	2	440	True
0.75	5.9000	5.9000	52	2	920	True
0.95	42.7000	42.7000	287	2	4670	True
0.99	226.6999	226.6999	1459	2	23370	True

Politica optima (VI) por gamma, para cada d (independiente de p):

gamma=0.5: d=0 -> STAY, d=1 -> UP
gamma=0.75: d=0 -> STAY, d=1 -> UP
gamma=0.95: d=0 -> STAY, d=1 -> UP
gamma=0.99: d=0 -> STAY, d=1 -> UP



Respuesta:

γ	$V_{VI}^*(s_0)$	$V_{PI}^*(s_0)$	Iter. VI	Iter. ext. PI	$\pi^*(d=0)$	$\pi^*(d=1)$
0.50	1.30	1.30	23	2	STAY	UP
0.75	5.90	5.90	52	2	STAY	UP
0.95	42.70	42.70	287	2	STAY	UP
0.99	226.70	226.70	1 459	2	STAY	UP

($s_0 = (1,0)$, piso 1 sin demanda pendiente. Ver gráfica sensibilidad_gamma.png generada por el código de arriba.)

Qué cambia con γ . El valor $V^*(s_0)$ crece de forma pronunciada y no lineal, como es esperable de la cota $R_{\max}/(1 - \gamma)$: pasar de $\gamma = 0.5$ a $\gamma = 0.99$ multiplica el horizonte efectivo por 50 y el valor por más de 170. El número de iteraciones de Value Iteration hasta $\theta = 10^{-6}$ también crece fuertemente ($23 \rightarrow 1459$), consistente con la cota geométrica $k \propto 1/\ln(1/\gamma)$ del Inciso 2 de la Task 2: la contracción se debilita al acercarse γ a 1. Policy Iteration, en cambio, se mantiene en

2 iteraciones externas para los cuatro valores de γ probados (aunque cada evaluación interna sí se alarga con γ , reflejada en `bellman_pi` creciente).

Qué NO cambia: la política. Para los cuatro valores de $\gamma \in \{0.5, 0.75, 0.95, 0.99\}$ la política óptima es exactamente la misma: STAY sin demanda, UP con demanda. Esto no es casualidad ni un rango de prueba insuficiente, es consecuencia directa de cómo está construida la recompensa de la Task 3:

- Con $d = 0$: STAY da -1 ; UP/DOWN dan -1.5 (paso más movimiento innecesario) **y** llevan a la misma distribución de sucesores que STAY (el piso no afecta ni la recompensa ni la transición de la demanda). Moverse está estrictamente dominado, para cualquier $\gamma > 0$.
- Con $d = 1$: STAY perpetúa la demanda con certeza y cobra -1 por paso indefinidamente; UP/DOWN cobran $+10$ de inmediato y resetean la demanda. Ninguna cantidad de paciencia descontada compensa acumular -1 por paso contra cobrar $+10$ ahora.

Como ninguna de las dos comparaciones depende de γ (son diferencias de recompensa inmediata sobre transiciones idénticas), **no existe ningún valor de $\gamma \in (0, 1)$ donde el comportamiento cualitativo cambie** en este MDP simplificado. Esto contrasta con la predicción de la Task 1 (Inciso 5), donde sí se esperaba un cambio cualitativo entre γ bajo (miope, atiende siempre lo más cercano) y γ alto (previsor, sacrifica demanda cercana por una emergencia lejana): ese fenómeno requiere que el modelo sepa *en qué piso* está la demanda, algo que el MDP de la Task 3 no representa. Es la primera evidencia concreta de la brecha entre el diseño completo y su simplificación, que se retoma en el Inciso 4.

1.7.4 Inciso 4

Conexión con el diseño propio: comparen la política óptima obtenida para el MDP simplificado con el comportamiento que esperarían del MDP que diseñaron en la Tarea 1. ¿Qué aspectos del dominio real no captura el MDP simplificado? ¿Cómo afectaría eso a la política óptima en el dominio real?

Respuesta:

El MDP simplificado reproduce solo la superficie del problema del Inciso 1 de la Task 1. Comparando variable por variable:

Diseño Task 1	Simplificación Task 3	Consecuencia
$\mathbf{u} \in \{0, 1\}^5$: llamada pendiente por piso	$d \in \{0, 1\}$: un solo bit global	El controlador no sabe hacia dónde moverse; cualquier movimiento “atiende” la demanda, como se vio en el Inciso 1 con el no-operación en el piso 5
$e \in \{0, 1\}^5$: emergencias con prioridad propia	No existe	No hay forma de preferir una emergencia lejana sobre una llamada ordinaria cercana
$\mathbf{v} \in \{0, 1\}^5$: destinos internos de pasajeros a bordo	No existe	El modelo no distingue “recoger” de “dejar”; asume un solo tipo de demanda

Diseño Task 1	Simplificación Task 3	Consecuencia
$d \in \{\text{sube, baja, detenido}\}$ (inercia)	Acción actual sin costo de reversión	No penaliza cambiar de sentido, así que no hay incentivo a completar un recorrido antes de invertir la marcha
Recompensa ponderada (w_t, w_e, w_c) por objetivo en tensión	Una sola función fija, sin componente de emergencia	No hay ponderación que ajustar; el trade-off tiempo-de-espera vs. energía vs. emergencia del Inciso 3 de la Task 1 no puede manifestarse

La política óptima de la Task 3 (“quédate si no hay demanda, muévete si la hay”) es razonable como heurística mínima, pero es degenerada frente al dominio real: satisface la función de recompensa sin garantizar que la cabina llegue al piso correcto, porque el estado no registra *dónde* está la llamada. En el edificio real esa política dejaría, por ejemplo, a una camilla esperando en el piso 5 mientras la cabina “atiende” moviéndose entre los pisos 1 y 2, porque para el modelo simplificado cualquier movimiento con $d = 1$ es indistinguible de otro.

Si se resolviera el MDP completo del Inciso 1, la política óptima debería en cambio: (a) enrutar la cabina hacia el piso específico con la llamada más urgente, ponderando distancia contra urgencia; (b) interrumpir un recorrido para atender una emergencia según el peso w_e , incluso a costa de dejar pendiente una llamada ordinaria cercana, un comportamiento que la Task 1 (Inciso 5) predijo dependiente de γ y que el Inciso 3 de esta tarea mostró que el modelo simplificado **no puede** exhibir bajo ningún γ ; y (c) evitar reversiones de sentido innecesarias por el costo de inercia. Ninguno de esos tres comportamientos es representable en el espacio de estados (p, d) de la Task 3.

1.7.5 Inciso 5

Dictamen final: redacten un párrafo dirigido a la gerencia del hospital argumentando si Programación Dinámica es una solución viable para el sistema de ascensores real, cuáles son sus limitaciones principales, y qué metodología recomendarían como paso siguiente.

Respuesta:

Dictamen para la gerencia. La Programación Dinámica es una herramienta válida para *este primer paso* del proyecto, no para el sistema final: sobre el modelo simplificado demostramos que Policy Iteration y Value Iteration convergen exactamente a la misma política óptima (Inciso 1), que la elección de algoritmo importa menos por el número de iteraciones que por cómo está implementada cada una (Inciso 2), y que a esta escala reducida el comportamiento óptimo resulta insensible al horizonte de planeación porque el modelo omite información de piso y de prioridad (Inciso 3 e Inciso 4).

El MDP realista de la Tarea 1, con cerca de 491 520 estados, ya está en el límite de lo que la Programación Dinámica tabular exacta puede resolver por memoria y tiempo, y un edificio con más pisos, más ascensores o más señales (ocupación de cabina, franjas horarias) lo haría inviable mucho antes de acercarse a ese tamaño. Además, la Programación Dinámica exige conocer de antemano las probabilidades de transición (tasas de llamada por piso y por hora), un supuesto de

estacionariedad que un hospital real rompe sistemáticamente por cambios de turno, estacionalidad y eventos imprevistos.

Recomendamos usar este modelo tabular como banco de pruebas y como política de referencia contra la cual comparar métodos que sí escalen — aproximación de funciones de valor o Aprendizaje por Refuerzo (por ejemplo Deep Q-Learning) entrenado sobre un simulador del edificio, combinado con descomposición jerárquica (decidir primero a qué piso ir y luego cómo llegar) para no sacrificar la interpretabilidad que la gerencia necesita antes de comprometer presupuesto en un sistema de aprendizaje completo.